# 26

# Real-time Gaming Leaderboard

In this chapter, we propose a design for a real-time gaming leaderboard. It shows a daily, weekly, and monthly leaderboard for a mobile game. The leaderboard ranks players based on their points earned playing the game. The higher a player's points are, the higher the player's position is on the leaderboard. We assume the game has 1 million daily active users (DAU).

| Term | Meaning |
|---|---|
| Leaderboard | A ranking system that displays the top-performing users in a game, based on their points or scores. |
| Tie | When two or more players have the same score, they share the same rank on the leaderboard. |
| Tie breaker | A rule used to rank players who have the same score, typically based on who achieved the score first. |
| Point | A numerical value assigned to a player's performance in a game, used to determine their ranking on the leaderboard. |

## Step 1 - Understand the Problem and Establish Design Scope

Leaderboard is a complex problem. It can be interpreted in many different ways. Ask questions to clarify requirements. Here is an example of a possible set of questions and assumptions.

**Question**: Is a tie allowed on the leaderboard?
**Assumption**: Yes, if two players have the same score, they share the same rank.

**Question**: Do we need to handle tie breaker?
**Assumption**: Tie breaker is not important for this interview.

**Question**: How many users are expected on the leaderboard?
**Assumption**: We expect 1 million daily active users (DAU) playing the game.

**Question**: Do we need to support friends' leaderboard?
**Assumption**: Yes, we only need to consider friends' leaderboard. Friends are defined as 1st degree connections.

**Question**: Do we need to support different types of leaderboards?
**Assumption**: Yes, we need to support daily, weekly, and monthly leaderboards.

**Question**: How do points earned from different types of actions affect the leaderboard?
**Assumption**: Each action earns a fixed number of points. For simplicity, assume all actions are worth 10 points.

**Question**: Do we need to support different types of points?
**Assumption**: Yes, we need to support different types of points, such as those earned from playing the game, sharing the game with friends, and inviting new users to the game.

**Question**: Do we need to support a global leaderboard?
**Assumption**: No, we only need to support a national leaderboard.

**Question**: Do we need to support a historical leaderboard?
**Assumption**: Yes, we need to support a historical leaderboard. For simplicity, assume it shows the leaderboard from the last month.

**Question**: How do we handle cheating or fraudulent activity?
**Assumption**: We can assume we have a separate system in place to detect cheating and fraudulent activity. The leaderboard does not need to handle this.

**Question**: Do we need to support real-time updates?
**Assumption**: Yes, we need to support real-time updates. The leaderboard should reflect the latest scores within a few minutes.

**Question**: Do we need to support a notification system?
**Assumption**: No, we do not need to support a notification system.

Here are the functions that are in scope:

- Daily, weekly, and monthly leaderboards
- Points are assigned based on different types of actions
  - Playing the game
  - Sharing the game with friends
  - Inviting new users to the game
- Friends' leaderboard
- National leaderboard

Here are the functions that are not in scope:

- Tie breaker
- Cheating and fraudulent activity
- Notification system
- Different types of points
- Global leaderboard
- Historical leaderboard

## Step 2 - Propose High-Level Design and Get Buy-in

### API design

We use RESTful API for client-server communication.

#### Post a score

This API adds a score to a user's total score on a specific leaderboard.

Request

```
POST /v1/scores
{
  "user_id": "mtan",
  "points": 10
}
```

Response

| Field | Description |
|---|---|
| user_id | The ID of the user who earns points. |
| points | The number of points earned in this action. |

```
HTTP 200 OK
{
  "user_id": "mtan",
  "total_score": 1510,
  "leaderboards": [
    {"type": "daily", "rank": 120},
    {"type": "weekly", "rank": 150},
    {"type": "monthly", "rank": 200}
  ]
}
```

#### Get leaderboard

Get the top 10 players from the leaderboard.

Request

```
GET /v1/leaderboard?type={type}&location={loc}
```

Response

```
HTTP 200 OK
{
  "type": "weekly",
  "location": "global",
  "last_updated": "2025-01-01T00:00:00Z",
  "top_players": [
    {"user_id": "user_1", "rank": 1, "points": 2000},
    {"user_id": "user_2", "rank": 2, "points": 1900},
    {"user_id": "user_3", "rank": 3, "points": 1800},
    ...
    {"user_id": "user_10", "rank": 10, "points": 1100}
  ]
}
```

#### Get relevant leaderboard

Get the user's current rank.

Request

```
GET /v1/leaderboard/relevant
```

Response

```
HTTP 200 OK
{
  "type": "weekly",
  "location": "global",
  "last_updated": "2025-01-01T00:00:00Z",
  "above": [
    {"user_id": "user_4", "rank": 4, "points": 1700},
    {"user_id": "user_5", "rank": 5, "points": 1600},
    {"user_id": "user_6", "rank": 6, "points": 1500},
    {"user_id": "user_7", "rank": 7, "points": 1400},
    {"user_id": "user_8", "rank": 8, "points": 1300},
    {"user_id": "user_9", "rank": 9, "points": 1200}
  ],
  "user": {"user_id": "mtan", "rank": 120, "points": 1510},
  "below": [
    {"user_id": "user_10", "rank": 10, "points": 1100}
  ]
}
```

### High-level architecture

The high-level architecture is shown below.

LB
Client
Cache
Cache
Cache